Smart Tongs

To Advise Waste-Free Serving Sizes

T3AM THR33

- Nikky Woeger
- Declan Otanez
- Jake Stewart
- David Gurung
- Max Goldenson

Problem

Food Waste

Our Solution

- Encouraging consumers to take only the portion of food they will consume.

- Establishment of a calculated system of food production.

Hence, Smart Tongs

Target Audience

The everyday diner at the various restaurants in the campus.

Design Constraints

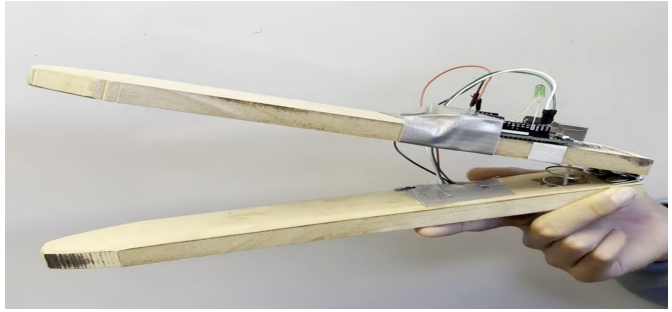
- All components must be resistant to temperatures 0-60 degrees Celsius
- Exposed components must be food safe
- Dimensions per each component must be of appropriate amounts
  - For example, hinge dimensions must be less than 60 millimeters in length, width and height
- Battery must power all electronics to last a 6 hour shift
- Tongs should be easy to use
- User feedback must be easily understood
- Non-discriminatory use

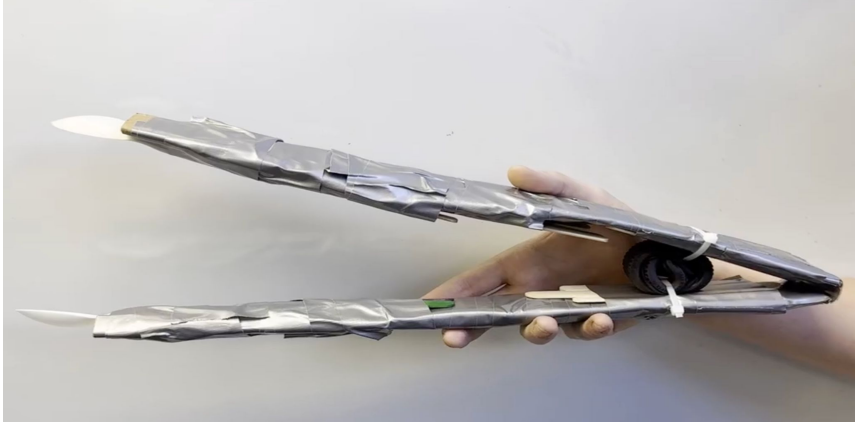
Design Constraints cont.

Design Constraints				
Name	**Dimensions (L, W, H) (mm)**	**Max-min temps (C)**	**Power Draw**	**Food/clean Safe**
Hinge	(<60, <60, <60)	0-60	no	yes
Hinge shaft	(>40, 8, 8)	0-60	no	yes
Magnet	(2, 4, 4)	0-60	no	no
Encoder	(23, 23, 3.5)	0-60	yes ( )	no
Grippers	(<20, <20, <30)	0-60	no	yes
Tong shafts	(200<x<300, 20<x<30, 20<x<30)	0-60	no	yes
PCB	(75, 20, 3.5)	0-60	yes ( )	no
Accelerometer	(<23, <23, <23)	0-60	yes ( )	no
Spring	(<12, <12, 90<x<100) .5<k<1.5	0-60	no	yes
Bearings	(7, 22, 22)	0-60	no	yes
Rumble Motors	11mm x 3mm (D x T)	0-60	yes ( )	no
Screws	M2 x 4, M4x4	0-60	no	yes
Batteries	(18, 18, 66)	0-61	yes ( )	no

Design Process

- Prototypes containing different components





- Find strengths and weaknesses of our ideas and designs
- Start to mix our different parts of the project for overall development

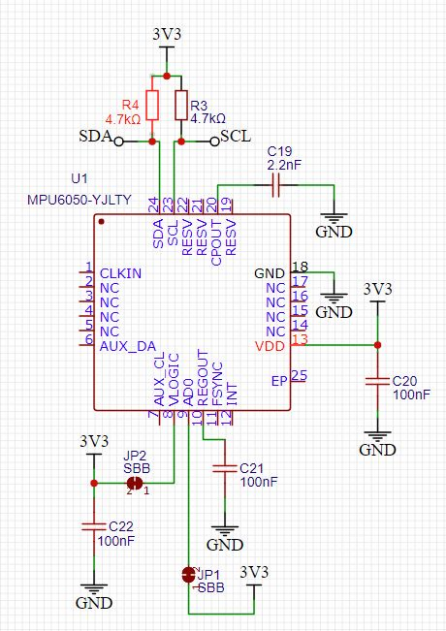
Central Electronics Unit

Electronics Unit Design Criteria:

Design Goal: ***Smart Tongs that Recommend Appropriate Serving Size***

Electronics Design Criteria:

- *Battery Must last a full 6-hour shift*
- *Sence New Users (90% Accuracy)*
- *Tolerate High Temperatures (< 60c)*
- *Interactive Multifaceted User Feedback*
- *IoT / BLE Enabled*



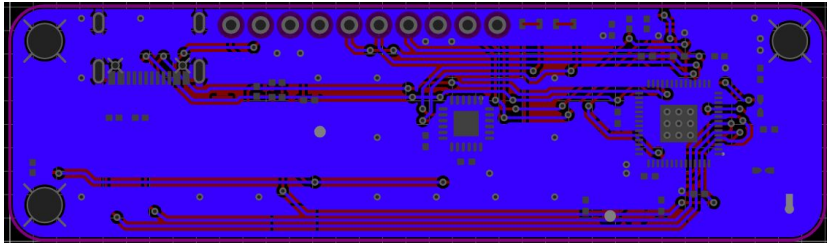
Electronics Unit Vision:

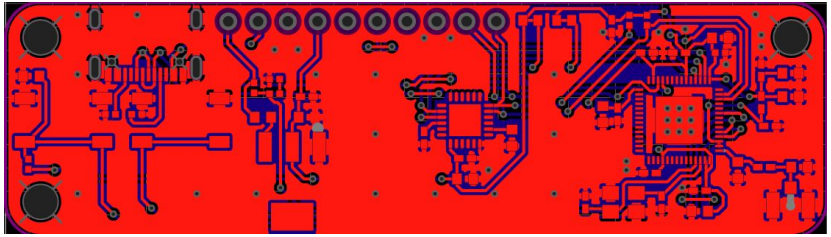
Design Criteria:

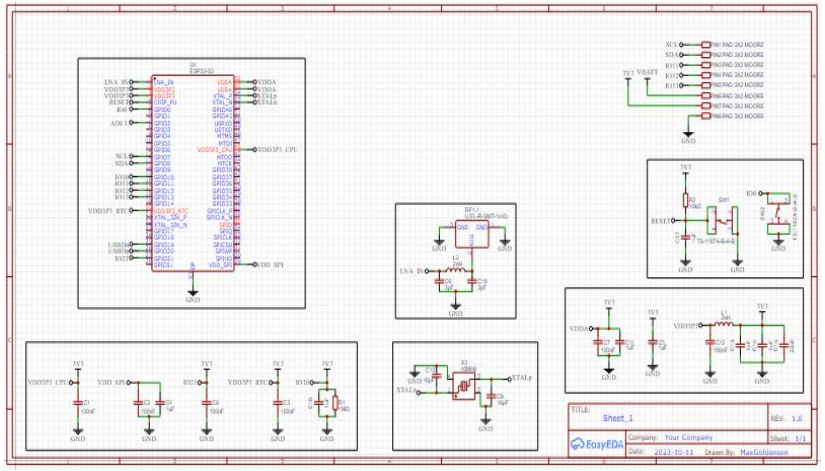
- *Battery Must last a full 6-hour shift*
- *Sence New Users (90% Accuracy)*
- *Tolerate High Temperatures (< 60c)*
- *Report Data Over Bluetooth*
- *Provide Tactile User Feedback*

We'll Use:

- *Accelerometer of User Detection*
- *Quadric Encoder for Tong Position*
- *ESP32-S3 for Wireless Capabilities*
- *Rumble Motor / Display*

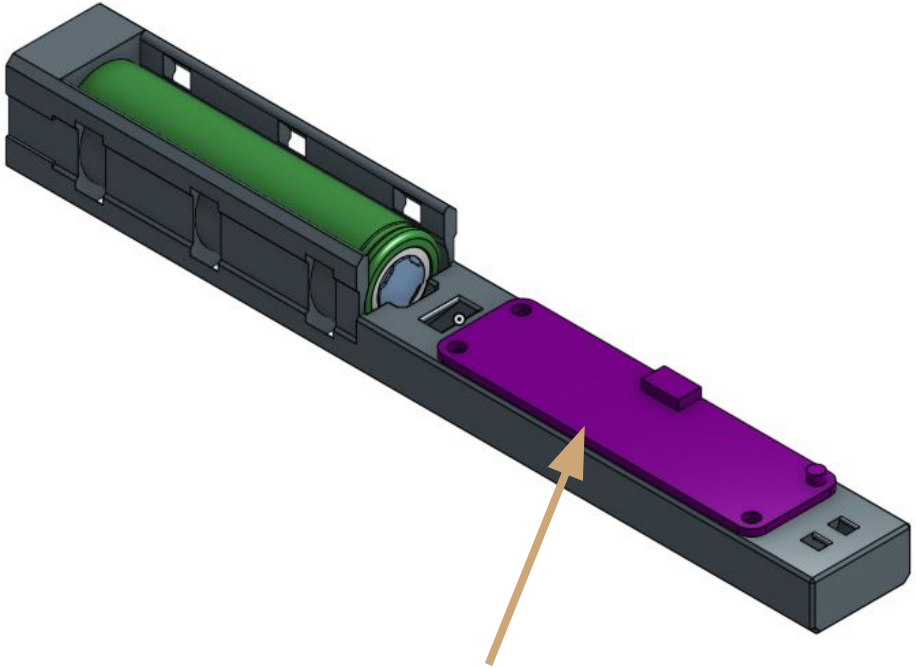


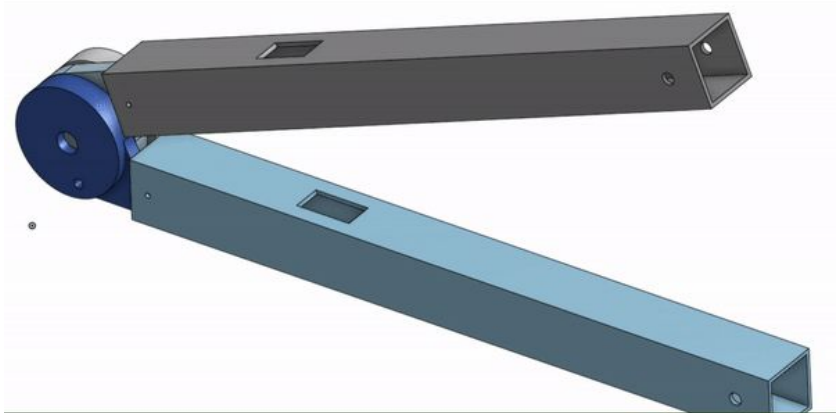


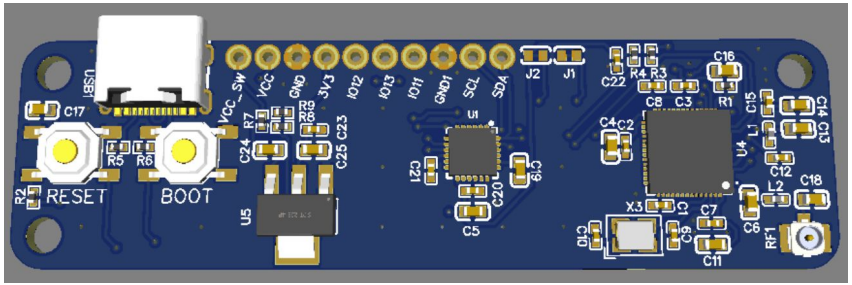


Execution:

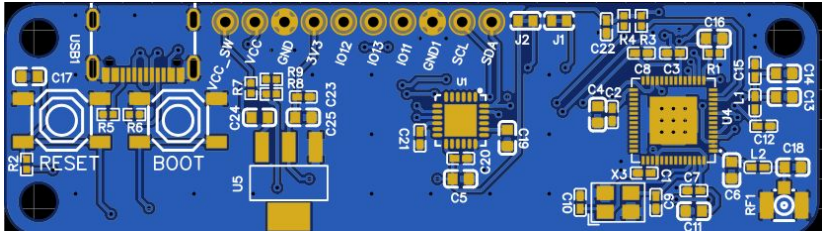
- ***Electronics Unit***
  - *Fits into the handle*
  - *Custom Circuit Board*
  - *High Temperature Tolerance*
  - *Food-Resistant Location*







Technical Analysis:

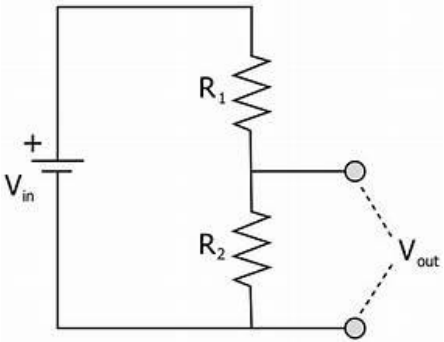


Power Consumption:

- *Theoretical 200mah Power Draw*
  - *Linear Regulator & 2000mah Battery*
  - *Approximately 10h Battery Life*

Voltage Divider:

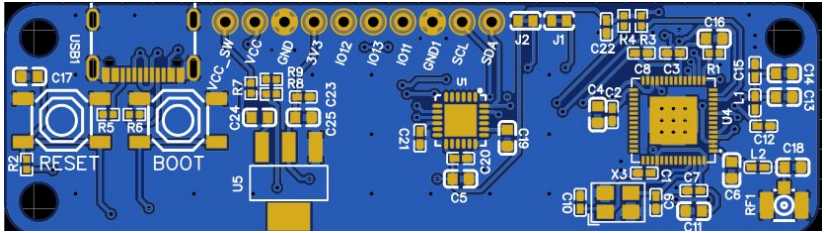
- *Microcontroller can measure it's battery*
  - *With minimal current draw*



Anticipated CPU Temps:

- *Through Cooling / Power values*
  - *From the ESP-32 / 18650 Datasheets*

Technical Analysis:



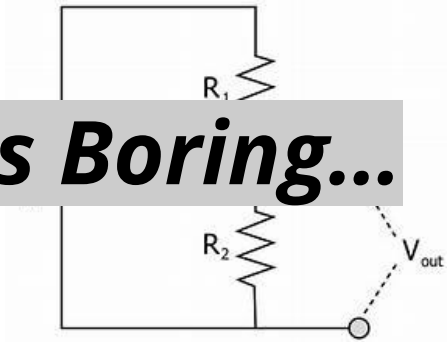
Power Consumption:

- *Theoretical 200mah Power Draw*
  - *Linear Regulator & 2000mah Battery*
  - *Approximately 10h Battery Life*

Voltage D

This Level of Analysis is Boring...

- *Microcontroller can measure it's battery*
  - *With minimal current draw*



Anticipated CPU Temps:

- *Through Cooling / Power values*
  - *From the ESP-32 / 18650 Datasheets*

Let’s look at something interesting…

Antenna Impedance Matching:

It's Called a Smith Chart And it's Helpful.

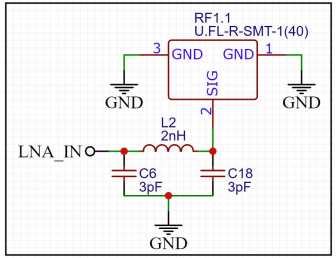
- *Appropriate Impedance Matching for the Antenna*
  - *Microcontroller has an impedance of (35 + 10i)Ω, convert to 50Ω*
  - *Resistance and Reluctance, represented as a complex number*
- *To Match Impedance, Meet at Center*
  - *Through Series and Parallel Capacitors / Inductors*
- *Recorded Translations Relate To Ideal Values*
  - *Validate Through Spectrometry Analysis*

$$K = \sqrt{(Zl / Zs)}$$

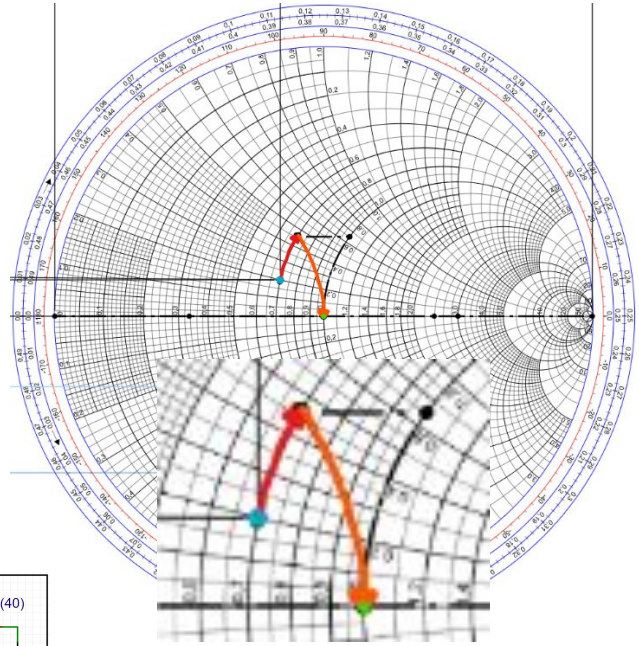
$$L = (Zs * K) / \omega$$

$$C1 = 1 / (Zs * \omega)$$

$$C2 = 1 / (Zl * \omega)$$



Smith Chart:



Delta Resistance 12.5 Ohms
Delta Reluctance 32.5 Ohms

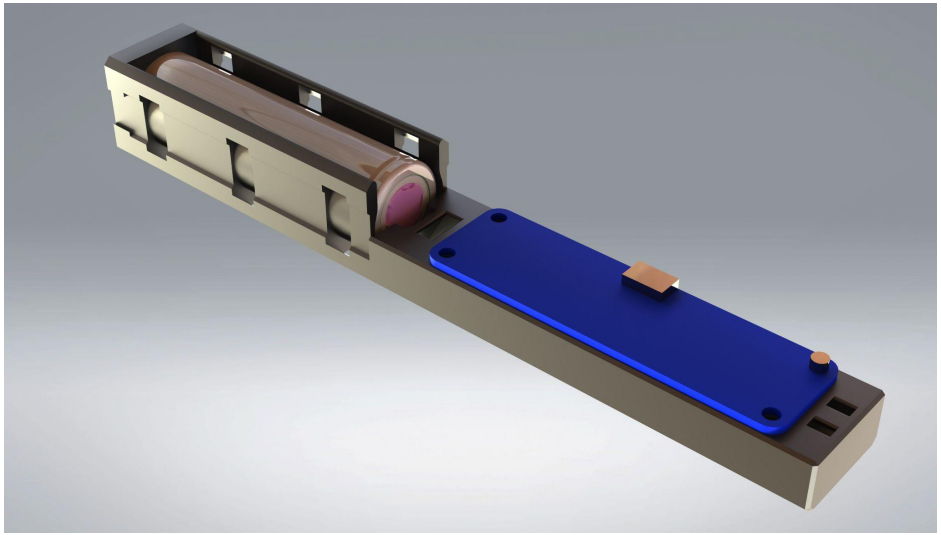
Physical Progress

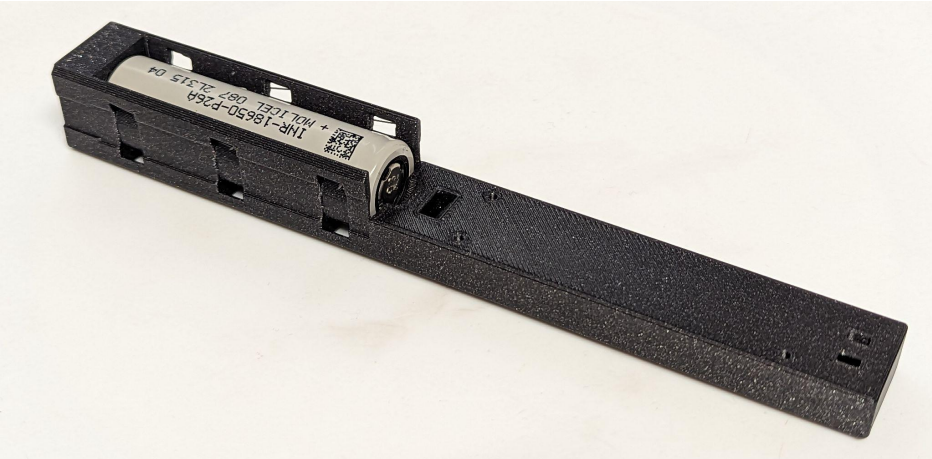
Completed:

- *3D-Printed Prototype Enclosure*
- *Schematic / Layout Compete*
- *Passed Design Rule Check*
- *Sourced battery*

What's Left:

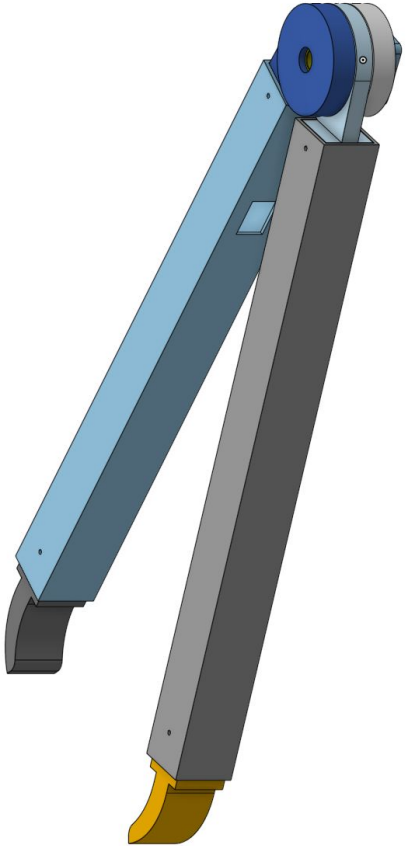
- *Order Custom PCBs*
- *Integrate With Other Subsystems*





Computer Aided Design

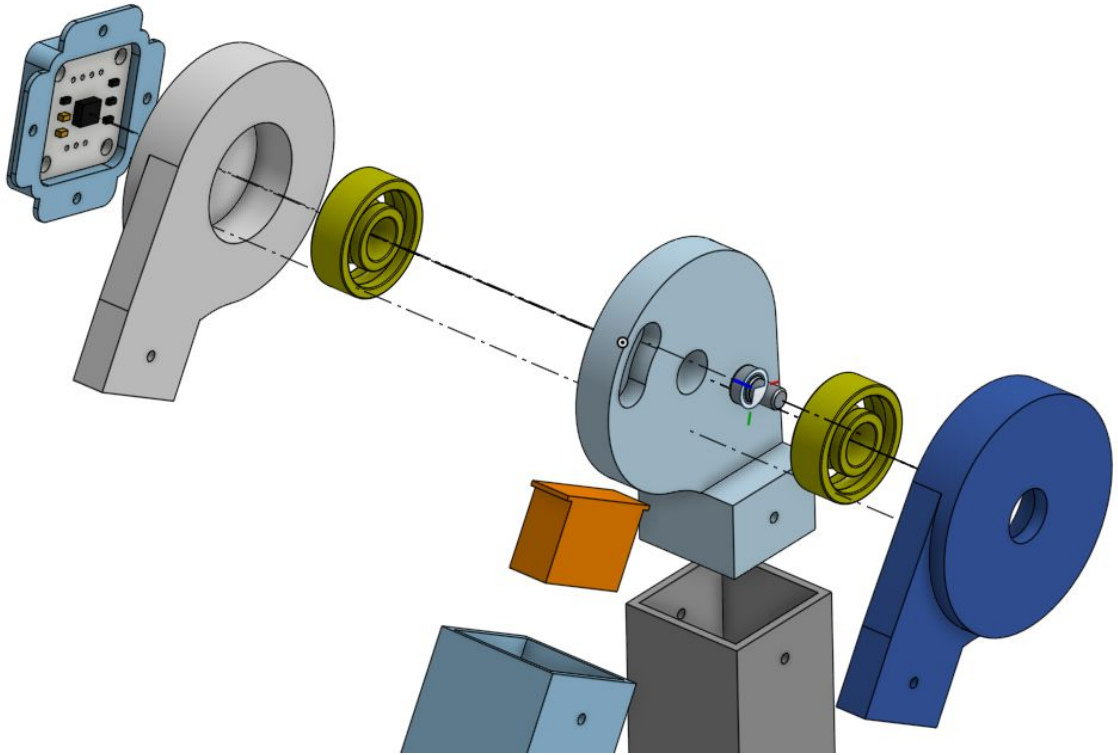
- Giving thoughts specificity
- Determining Materials and Dimensions
- Allowed for Planning
- Purchase Materials and designed around them
- Integrates all physical parts
    - See how they work together
- Accurate and well-made prototypes

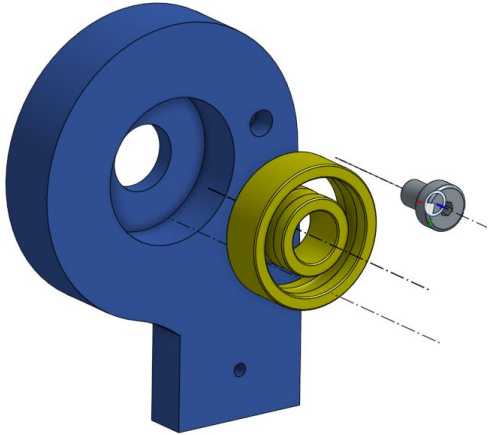


List of Essential Parts

Part Name:	Quantity:	Material:	Bought/Manufactured?:	PPU:
Tong Sides	2	Aluminum 6061	Manufactured	$1.61
Back Hinge	1	ABS	Manufactured	$0.00
Middle Hinge	1	ABS	Manufactured	$0.00
Front Hinge	1	ABS	Manufactured	$0.00
Electronics Holder	1	ABS	Manufactured	$0.00
Bearing Insert	1	ABS	Manufactured	$0.00
Hinge Lock Bolt	1	Steel	Bought	(Consumable)
Rotary Encoder	1	Mixed	Bought	(Consumable)
Dipole Magnet	1	Mixed	Bought	(Consumable)
Hinge Screws	4	Steel	Bought	(Consumable)
Battery	1	Mixed	Bought	$3.99
Accelerometer	1	Mixed	Bought	$3.33
Tong ends	2	ABS	Manufactured	$0.00
Spring mounts	2	ABS	Manufactured	$0.00

Exploded View of Hinge

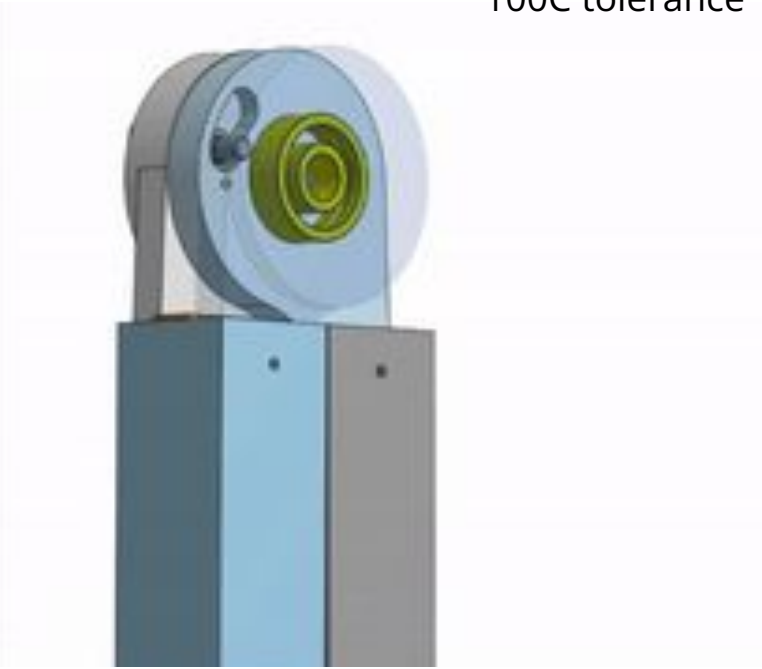




Critical Design Components

Hinge:

100C tolerance



Spring:

- Ordered from McMaster.com
- Using the CAD design

$$F = -k * x$$

44.4 N, –0.045 m

44.4 N = -k * -0.045 m

k = 986.7 Newtons/meter

k = .987 N/mm

User Testing to test for optimum force

Critical Design Components

Software

- Accelerometer (measure movement of device)
  - Takes in x, y, and z directions
- User feedback code
- Measure angle of tongs to determine when used
- Arduino code developed in VS code

```
tongs.cpp > setup(void)
#include <Adafruit_MPU6050.h>
#include <Adafruit_Sensor.h>
#include <Wire.h>
#include "AS5600.h"
#include "Wire.h"

Adafruit_MPU6050 mpu;

void setup(void)
{
    pinMode(10, OUTPUT);
    Serial.begin(115200);
    while (!Serial)
        delay(10); // will pause Zero, Leonardo, etc until serial console opens

    Serial.println("Adafruit MPU6050 test!");

    // Try to initialize!
    mpu.begin();

    delay(1000);

    mpu.setAccelerometerRange(MPU6050_RANGE_8_G);

    mpu.setGyroRange(MPU6050_RANGE_500_DEG);

    mpu.setFilterBandwidth(MPU6050_BAND_21_HZ);

    Serial.println("");
    delay(100);

    AS5600 as5600;
    Wire.begin();
}

void loop()
{

    /* Get new sensor events with the readings */
    sensors_event_t a, g, temp;
    mpu.getEvent(&a, &g, &temp);

    /* Print out the values */
    Serial.print("Acceleration X: ");
    Serial.print(a.acceleration.x);
    Serial.print(", Y: ");
    Serial.print(a.acceleration.y);
    Serial.print(", Z: ");
    Serial.print(a.acceleration.z);
    Serial.println(" m/s^2");

    Serial.println("");
    delay(500);

    double totalAcceleration;
    bool tongs = true;
    bool program = true;
    bool click = true;
    totalAcceleration = (((a.acceleration.x) ^ 2) + ((a.acceleration.y) ^ 2) + ((a.acceleration.z) ^ 2));

    while (program == true)
    {
        if (totalAcceleration < 0.9 || totalAcceleration > 1.1)
        {
            tongs = true; // true means in use
        }
        else
        {
            tongs = false; // tongs are not in use
        }

        if (tongs = true) // when they are in use
        {
            void displayScoops(int scoopCount)
            {
                u8g2.clearBuffer();
                int centerCoordinates = u8g2.getDisplayHeight() / 2 + u8g2.getAscent();
                char y_str[20];
                snprintf(y_str, sizeof(y_str), "%d", scoopCount);
                String messages = "Scoops right now : " + String(y_str);
                const char *messages_cstr = messages.c_str();
                u8g2.drawStr(0, centerCoordinates, messages_cstr);
                u8g2.sendBuffer();
            }
            Serial.print(as5600.readAngle());

            if (click == true)
            {
                for (int i = 0; i < 3; i++)
                {
                    digitalWrite(10, High);

                    if (i > 3)
                    {
                        // print message after 3 clicks
                    }
                }
            }
            else
            {
                return false;
            }
            program = true;
        }
    }
}
```

Accelerometer Data Code

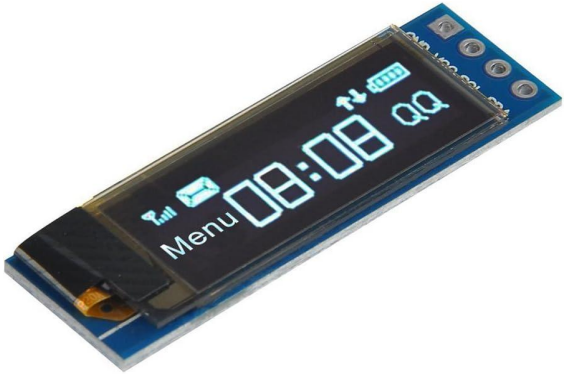
- Takes in information from accelerometer
- Checks if they are in use based off the result

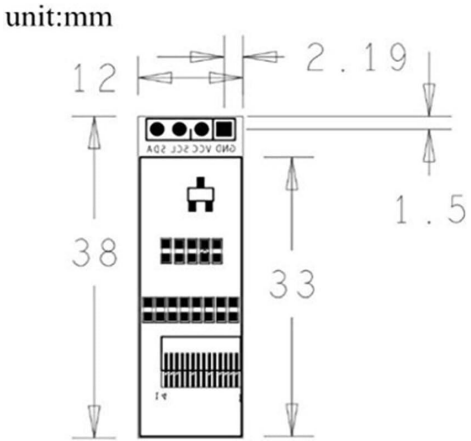
```
totalAcceleration = sqrt(pow(a.acceleration.x, 2) + pow(a.acceleration.y,2) + pow(a.acceleration.z,2));

if (totalAcceleration < 0.9 || totalAcceleration > 1.1)
{
    tongs = true; // true means in use
}
else
{
    tongs = false; // tongs are not in use
}
```

Critical Design Components

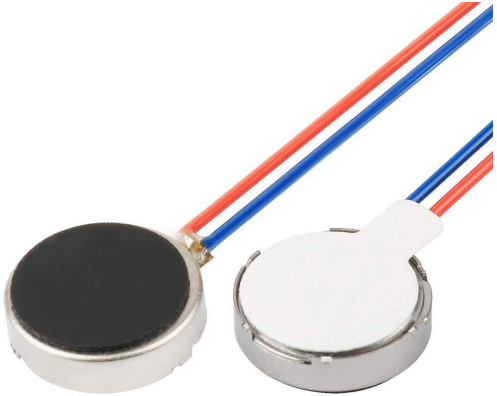
User Feedback





- **SSD1306, 128 x 32 Display, 3.3V/5V Serial OLED Programmed using U8g2 library**

- **3V Vibrating DC motors, 12000 RPM, 10MM x 3MM**



Manufacturing Process

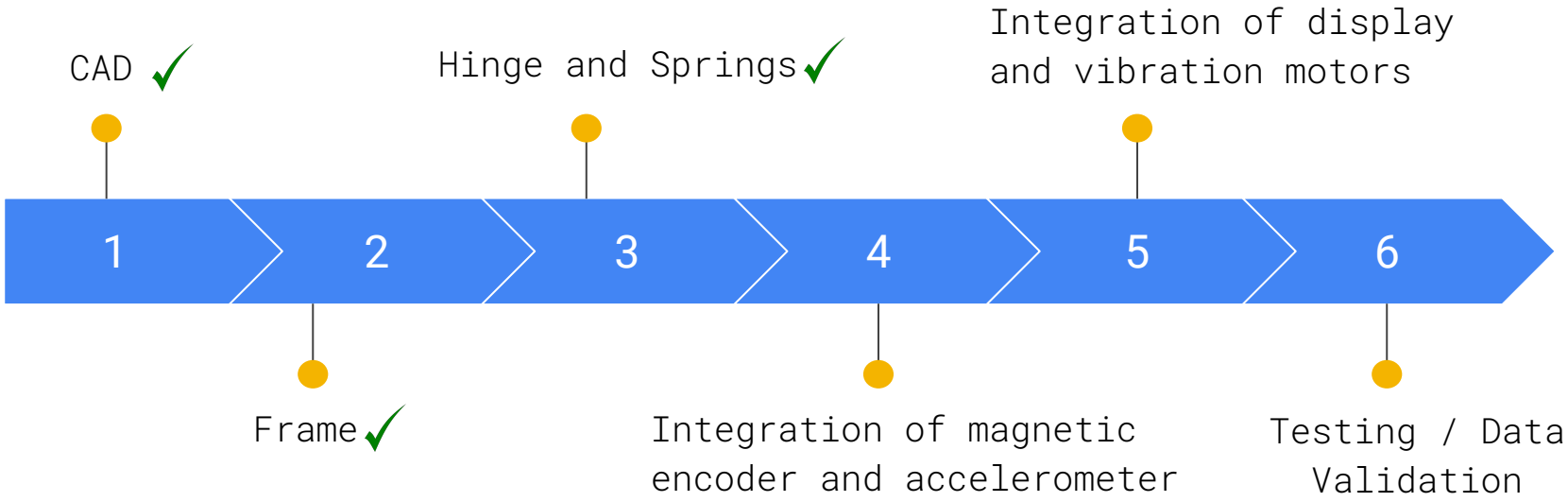
- Cut the bars to size
- Found center points
- Drilled holes for the hinge and tong grips
- Used CNC to make a profile
- Filed down the edges
- Lathe hinge rod

Plans for Attaching the Spring

- Drill a hole for a screw
- Thread the hole with a tap
- Put slotted washer over the spring
- Screw down the washer
- File down the other end of the screw that sticks out

Spring keeps the tongs from over extending

Plan for completion



Any questions?

Talk to us after class if you have any further questions.